

Universidade do Minho
Mestrado em Engenharia Informártica

Course Unit

Computing Systems Security

Academic year 2026/2027

Block 2 Report

Lab 3 — PAM and the authentication stack · Lab 4 — OIDC and federated identity · Lab 5 — SSH certificates, authorisation and hardening

Work by

Marco António Ferreira Abreu — PG64003

Lecturer: Vítor Francisco Fonte

November 2026

Contents

Introduction **4**
1. Lab 3 — PAM and the authentication stack **5**
1.1 Preparation questions 5
P1. The four PAM module types 5
P2. The shared file included by sudo 5
1.2 Part A: Who is a process? Credentials and NSS 5
Exercise 1a 5
Exercise 2a 6
Exercise 3a 6
Exercise 3b 7
1.3 Part B: The PAM stack 7
Exercise 4a 7
Exercise 4b 8
Exercise 4c 9
Exercise 4d 9
1.4 Part C: Break it, then repair it 9
Exercise 5a 9
Exercise 5b 10
1.5 Part D: Hardening the stack 10
Note: pam_faillock 10
Exercise 6a 11
Exercise 7a 12
2. Lab 4 — OIDC and federated identity **15**
2.1 Preparation questions 15
P1. The dex discovery document 15
P2. What the ID token proves 15
P3. The redirectURIs entry registered for democlient 15
2.2 Part A: The actors and the flow 16
Exercise 1a 16
2.3 Part B: Run the flow and capture the tokens 17
Exercise 2a 17
Exercise 2b 18
Exercise 2c 21
2.4 Part C: What the tokens prove 22
Exercise 3a 22
Exercise 3b 24
Exercise 3c 25
2.5 Part D: Misconfiguration 1, redirect_uri validation 25
Exercise 4a 25
Exercise 4b 27
Exercise 4c 28

Exercise 4d 29
2.6 Part E: Misconfiguration 2, failing to validate the ID token 29
Exercise 5a 29
Exercise 5b 30
Exercise 5c 31
3. Lab 5 — SSH certificates, authorisation and hardening 32
3.1 Preparation questions 32
P1. Running containers and generated keys 32
P2. Two operational problems of plain public keys 32
3.2 Part A: Key-based authentication and its limits 33
Exercise 1a 33
3.3 Part B: An SSH certificate authority: authentication, authorisation, revocation 34
Exercise 2a — User certificate 34
Exercise 3a — Principals and accounts 35
Exercise 4a — Authorisation model by principals 37
Exercise 4b — ops certificate: authentication and authorisation 39
Exercise 5a — Revocation with a KRL 40
Exercise 5b — The four-way distinction 41
3.4 Part C: Hardening sshd 42
Exercise 6a — Effective configuration, counts and the password test 42
Exercise 6b — Ranking the directives 46
Exercise 6c — PasswordAuthentication enabled 47
3.5 Part D: The agent-forwarding hijack 47
Exercise 7a — The attack 47
Exercise 8a — Mitigation 48
Appendix — Reflections 50
A.1 Lab 3 reflection 50
A.2 Lab 4 reflection 50
Note: fixing the state storage in client.py 52
A.3 Lab 5 reflection 53

Introduction

In this report I present all the work I carried out in the three labs of Block 2 of the course, devoted to authentication and authorisation. In Lab 3 I studied the local authentication stack of a Linux machine (process credentials, NSS and PAM); in Lab 4 I studied federated identity with OpenID Connect, using dex as the identity provider and a demo client; and in Lab 5 I worked with SSH on two containers, jump and target, moving from plain public keys to an SSH certificate authority, with authorisation by principals, revocation with a KRL, sshd hardening and the agent-forwarding hijack.

Unlike the assessed Block 2 report, which only included the required deliverables, this report gathers all my answers from the three labs: the preparation questions and every exercise of each part, with the configuration evidence I collected and the commands I ran. In the appendix, I include the reflections of the three labs. The screenshots were taken by me on the ssc-lab VM while doing the labs, except for Figures 37 and 47, which I took after the lab.

1. Lab 3 — PAM and the authentication stack

1.1 Preparation questions

P1. The four PAM module types

PAM has four types of modules: auth, account, password and session. auth proves the identity of the user, account checks whether the account can be used at this moment, password governs the credentials and, finally, session prepares the session (limits, environment, logs).

P2. The shared file included by sudo

The shared file from which /etc/pam.d/sudo pulls its auth rules is /etc/pam.d/common-auth, through the line @include common-auth.

```
student@ssc-lab:~/ssc-lab5$ cat /etc/pam.d/sudo
#%PAM-1.0

# Set up user limits from /etc/security/limits.conf.
session    required   pam_limits.so

session    required   pam_env.so readenv=1 user_readenv=0
session    required   pam_env.so readenv=1 envfile=/etc/default/locale user_readenv=0

@include common-auth
@include common-account
@include common-session-noninteractive
```

Figure 1 – Contents of /etc/pam.d/sudo, with the @include lines for the common-* files.

1.2 Part A: Who is a process? Credentials and NSS

Exercise 1a

On the Uid line there are four equal numbers, but each one represents something different. The first is the real UID, that is, the identity the session started with. The second is the effective UID, that is, the identity in use at this moment, which is the one used for permission checks. The third is the saved UID, which stores an identity the process can return to, allowing it to drop privileges temporarily and regain them later. The fourth number is the filesystem UID, which is used in file permission checks and, in practice, follows the same identity as the effective UID.

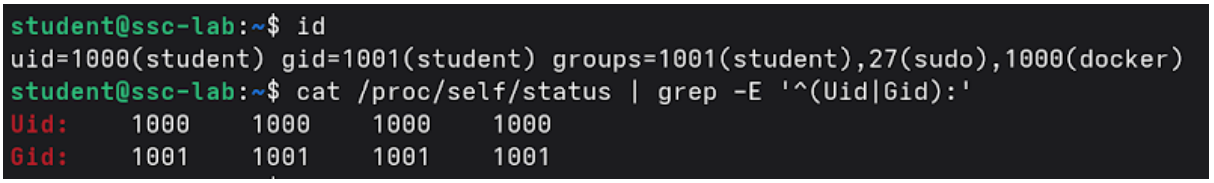


Figure 2 – Output of id and the Uid and Gid lines of /proc/self/status.

Exercise 2a

According to /etc/nsswitch.conf, NSS consults the sources files and systemd, that is, it first looks for the user in the local files and, if it finds nothing, looks in systemd. Therefore, the lookup is done locally.

If the accounts were stored in a directory service on another machine, the trust boundary of authentication would no longer be only this machine and would also include the directory service, the network over which the data travels and whoever administers that service. This has two consequences. First, if the directory service fails, no account stored in it will be able to log in to any machine. Second, if the service is compromised, the attacker can create or modify accounts and log in to every machine that trusts the directory service.

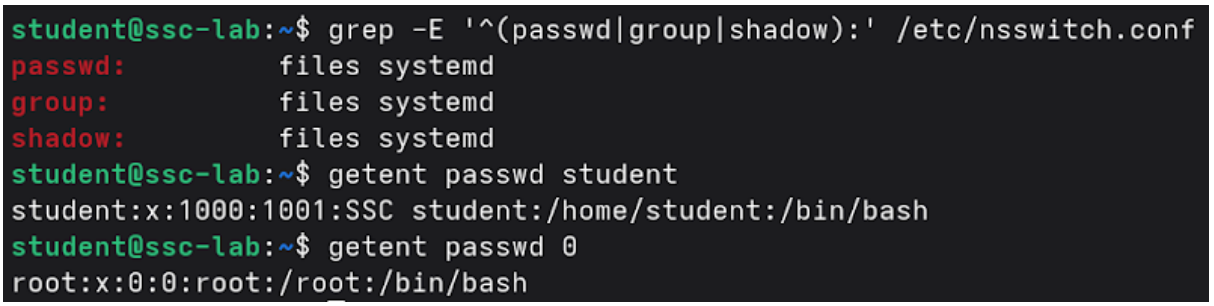


Figure 3 – NSS sources in /etc/nsswitch.conf and account lookups with getent passwd.

Exercise 3a

The /etc/passwd file is world-readable because programs that run without privileges may need to know the name of an account. To do so, they take the UID stored in the kernel, look it up in this file and obtain the name. For example, in the output of the command “getent passwd svc-report”, 997 is the UID and 988 is the GID. UID 997 is converted into the account name through /etc/passwd. Moreover, this file contains no secret, since the hash has been removed, as the “x” in the second field shows.

The /etc/shadow file is not world-readable, since it contains the users' password hashes; this way, an attacker with an unprivileged foothold cannot extract these hashes and work on them offline.

```
student@ssc-lab:~$ getent passwd svc-report
svc-report:x:997:988::/opt/report:/usr/sbin/nologin
student@ssc-lab:~$ sudo getent shadow student | cut -d: -f1,2
[sudo] password for student:
student:$y$j9T$hKsIdiOdHMx.Ur7GT3haz.$u.4QBMPnMHgAbLrro5iJEUv2KCnF4JBfG4kNCbLNk40
```

Figure 4 – getent passwd svc-report and the hash field of student in /etc/shadow.

Exercise 3b

The attack that keeping the hashes out of /etc/passwd frustrates is the offline brute-force attack on passwords. Keeping the hashes out of /etc/passwd prevents an unprivileged attacker from copying them and attacking them offline, on their own machine, with no limit on attempts, no account lockout and no logs. If the hashes were in /etc/passwd, readable by everyone, the attacker would copy the hash and could learn the hash function (yescrypt), the cost parameters (j9T), the salt (the field hKsIdiOdHMx.Ur7GT3haz.), and the rest would be the hash (u.4QBMPnMHgAbLrro5iJEUv2KCnF4JBfG4kNCbLNk40). With this, it would be enough to test every possible password combination, or lists of common passwords, until obtaining this hash. Since /etc/shadow is only readable with privileges, this attack is out of reach of an unprivileged attacker.

1.3 Part B: The PAM stack

Exercise 4a

In /etc/pam.d/sudo, three lines come from the common-* files: @include common-auth, @include common-account and @include common-session-noninteractive.

```
student@ssc-lab:~$ cat /etc/pam.d/sudo
#%PAM-1.0

# Set up user limits from /etc/security/limits.conf.
session    required   pam_limits.so

session    required   pam_env.so readenv=1 user_readenv=0
session    required   pam_env.so readenv=1 envfile=/etc/default/locale user_readenv=0

@include common-auth
@include common-account
@include common-session-noninteractive
student@ssc-lab:~$ cat /etc/pam.d/common-auth
#
# /etc/pam.d/common-auth - authentication settings common to all services
#
# This file is included from other service-specific PAM config files,
# and should contain a list of the authentication modules that define
# the central authentication scheme for use on the system
# (e.g., /etc/shadow, LDAP, Kerberos, etc.).  The default is to use the
# traditional Unix authentication mechanisms.
#
# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules.  See
# pam-auth-update(8) for details.

# here are the per-package modules (the "Primary" block)
auth    [success=1 default=ignore]      pam_unix.so nullok
# here's the fallback if no module succeeds
auth    requisite                       pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
auth    required                        pam_permit.so
# and here are more per-package modules (the "Additional" block)
auth    optional                        pam_cap.so
# end of pam-auth-update config
```

Figure 5 – Contents of /etc/pam.d/sudo and /etc/pam.d/common-auth.

Exercise 4b

pam_unix.so receives the user's password through a service and then reads this user's hash from /etc/shadow, through NSS, and computes the hash of the password it received with the same parameters (algorithm, salt and cost parameters). It then compares the two hashes. If it succeeds, it skips pam_deny.so and goes to pam_permit.so, which then continues to pam_cap.so, which is optional. However, if the hash it computed differs from the one in /etc/shadow, pam_unix.so fails and the stack continues to pam_deny.so, which denies authentication.

Exercise 4c

required is used to demand something without revealing to the user or to an attacker what went wrong: if it succeeds, it continues to the next line; however, if it fails, it marks the stack as failed, but keeps running until the end and then denies.

sufficient is used so that, if it passes, nothing else is needed: if it succeeds, it stops and accepts, unless an earlier required has failed; however, if it fails, it ignores the failure and continues.

In short, the difference is that the failure of a required dooms authentication, whereas the failure of a sufficient is ignored and the stack continues, that is, it does not doom authentication. The success of a required lets the stack continue, whereas the success of a sufficient stops the stack.

Exercise 4d

On the pam_unix.so line, the action [success=1 default=ignore] means that, if pam_unix.so succeeds, it skips one line and goes to pam_permit.so and then to pam_cap.so. Otherwise, it ignores the result of pam_unix.so and goes to the next line, pam_deny.so, which denies authentication.

If we had sufficient instead of [success=1 default=ignore], then, if the pam_unix.so module succeeded, sufficient would stop the stack and accept authentication. However, if pam_unix.so did not succeed, the failure would be ignored and the stack would move on to the next line, the pam_deny.so module, where authentication would be denied and, since it is marked as requisite, the stack would stop immediately and deny authentication.

So the difference lies in the success case: with the action [success=1 default=ignore], the stack goes from pam_unix.so to pam_permit.so and, finally, to pam_cap.so, whereas with sufficient it stops the stack at pam_unix.so.

1.4 Part C: Break it, then repair it

Exercise 5a

The error was:

sudo: PAM authentication error: Module is unknown

sudo: a password is required

When PAM cannot load a required module, it denies authentication and prints a message saying that there is an unknown module. This is the correct behaviour for an authentication stack since, when in doubt, the stack should fail closed (deny) and not fail open (accept). This is because the cost of denying is unavailability, while the cost of accepting would be an intrusion.

However, in the opposite scenario, that is, if the authentication stack accepted authentication when a required module could not be loaded (fail open), any configuration error, or a deleted or corrupted module, would let anyone authenticate without verification and obtain root privileges through sudo.

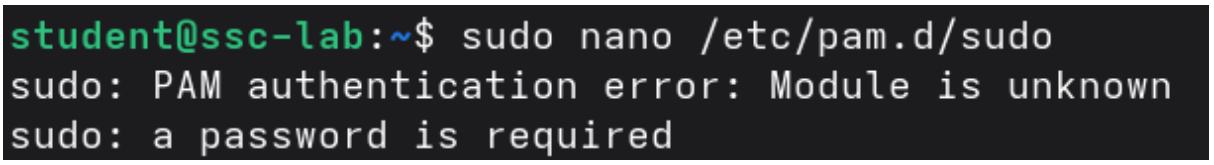


Figure 6 – sudo failing with “Module is unknown” after the broken edit.

Exercise 5b

This experiment was safe because we only changed /etc/pam.d/sudo and not common-auth, so login and sshd kept working normally. In addition, we had a terminal open as root, which allowed us to undo the damage even without access to sudo, as well as a snapshot of the machine and a copy of the old configuration. This teaches that, before starting to change authentication configuration, we should have a terminal open as root, a backup of the configuration and a snapshot of the machine, and we should start with the service with the least impact.

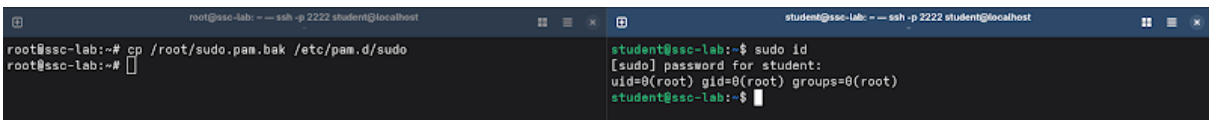


Figure 7 – Restoring /etc/pam.d/sudo from Terminal R and confirming that sudo id works again.

1.5 Part D: Hardening the stack

Note: pam_faillock

pam_faillock locks the account after several failed authentications. It has to wrap pam_unix.so, so in /etc/pam.d/sudo the @include common-auth line is replaced by the lines written out in full: if it were placed around the include, pam_deny.so would stop the stack before authfail could count the failure.

The account is locked if there are at least deny (3) failures within fail_interval (900 s) and the last one happened less than unlock_time (120 s) ago. The values go on every line because each line is an independent call to the module. preauth denies locked accounts, authfail records failures and authsucc clears the count. During the lockout, not even the correct password works.

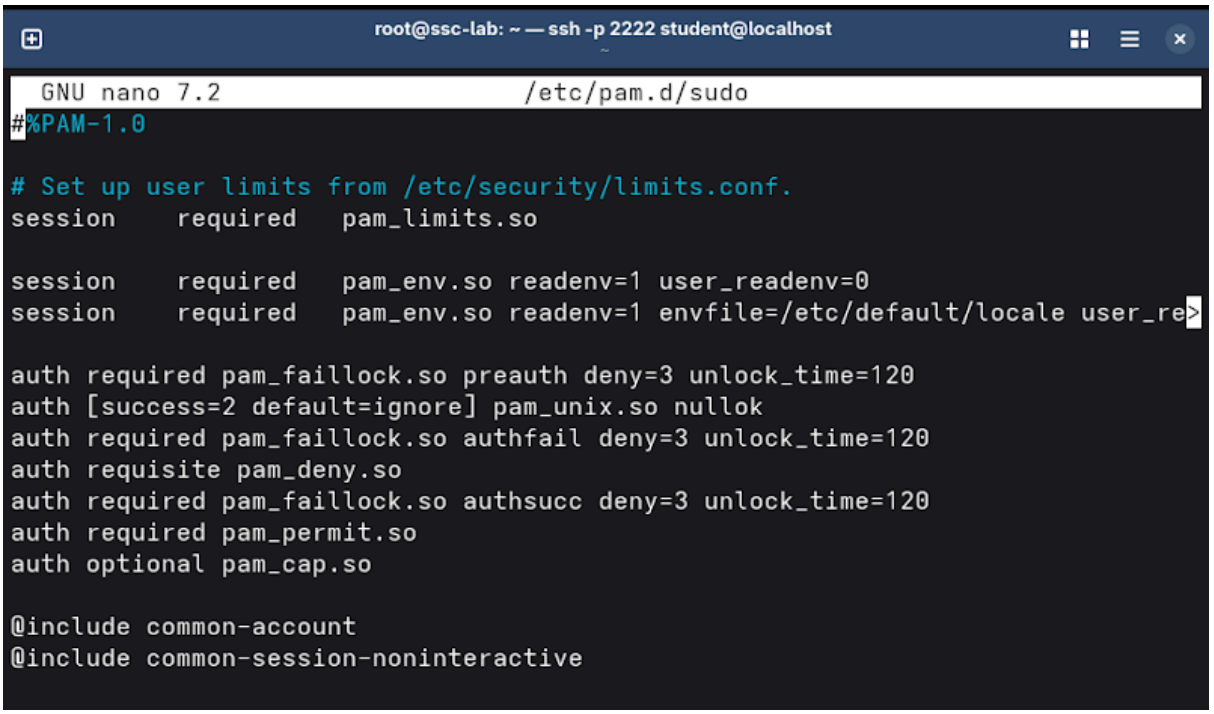


Figure 8 – /etc/pam.d/sudo with the pam_faillock lines written out in full.

Exercise 6a

Now, an attacker trying to guess passwords online has to wait 2 minutes after every 3 failures before trying again, that is, about 90 attempts per hour. This makes the attack very slow, which, in practical terms, makes it infeasible against reasonable passwords.

However, this lockout can become a threat to the machine and its legitimate users since, if the attacker does this constantly, the user is prevented from accessing the service. In this lab, it would prevent the user from using sudo.

Moreover, if this were in common-auth, it would prevent access to any account on the machine, and the attacker would only need to know the user's name. We would therefore have a denial of service.

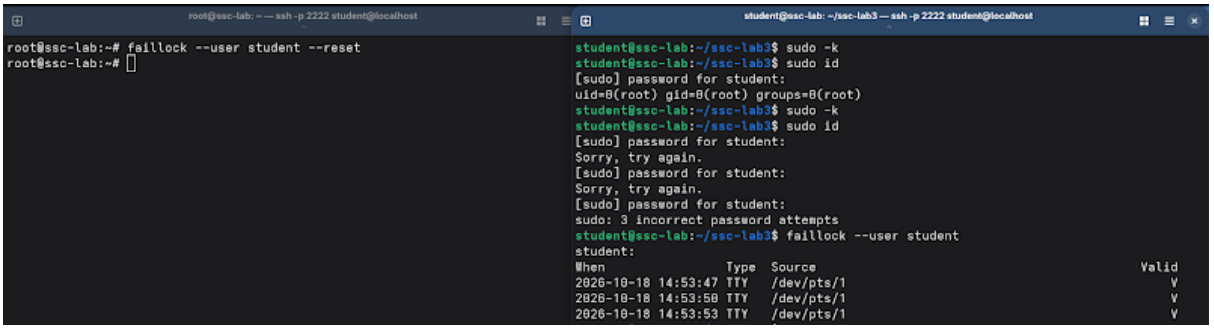


Figure 9 – Failed sudo attempts and the failure count shown by faillock.

Exercise 7a

The rejection message was BAD PASSWORD: The password contains less than 1 digits, triggered by the rule dcredit = -1, which I set in /etc/security/pwquality.conf. The test password was estudante, which the Ubuntu default accepts (it has 9 characters, above the minimum of 8), but which fails the three conditions of the rule defined in the lab: it has fewer than 12 characters, no digits and no uppercase letters. The message only reports the missing digit because the module reports the first rule that is not met.

The password-quality policy sits in the password type, and not in auth, since the auth type is used to authenticate users and not to govern credentials. The event that the password stack gates is the password change, that is, the choice of a new credential. This is different from a login attempt, in which no credential is created: identity is proven with a password that already exists. Furthermore, it would make no sense to have the quality check during login, since login is used to compare the password with the hash, and if the quality check were done there it could make the stack fail, which would mean the user could not log in. Therefore, the password-quality check must be done before the password is set.

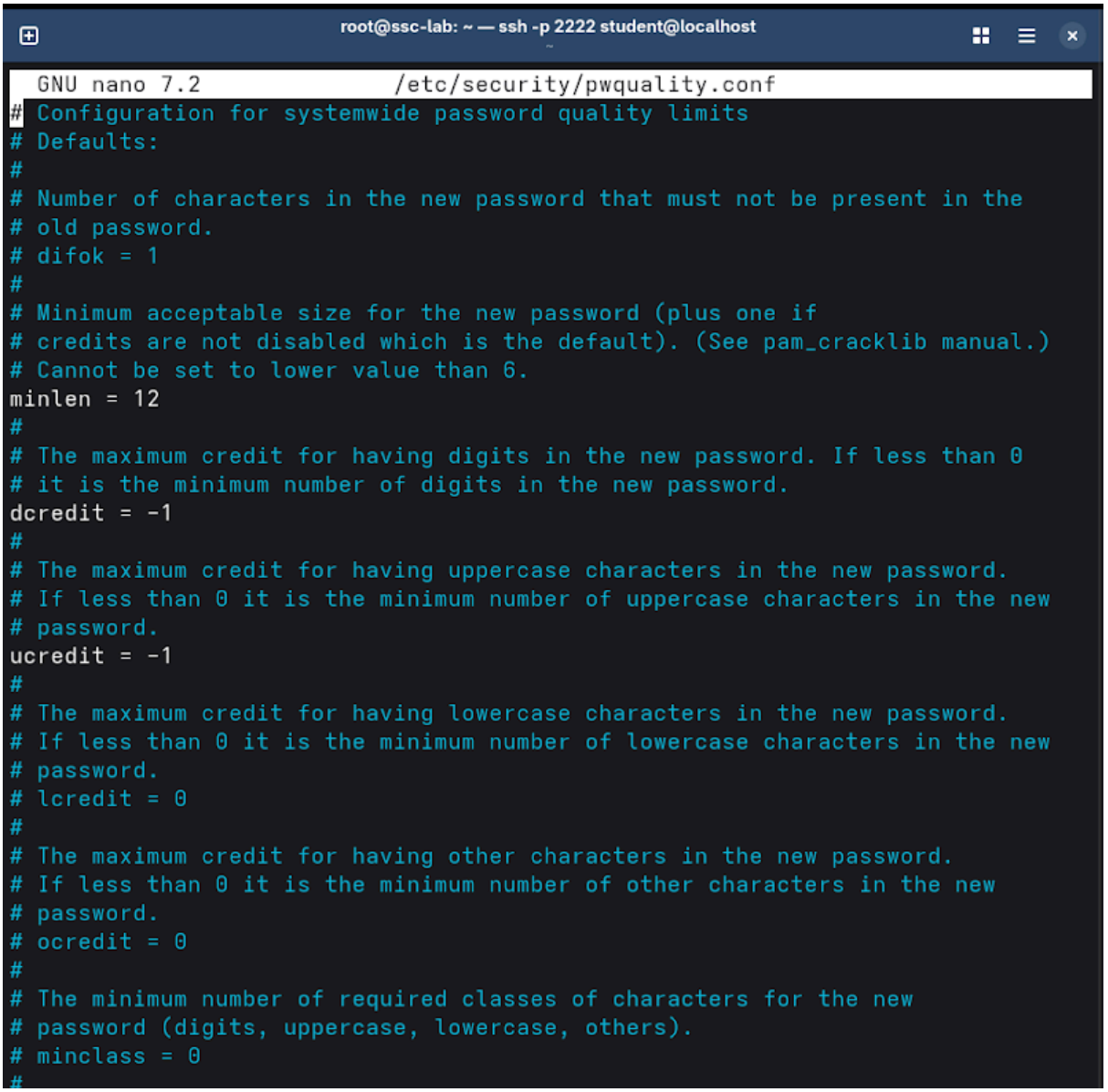


Figure 10 – Password-quality rules in /etc/security/pwquality.conf.

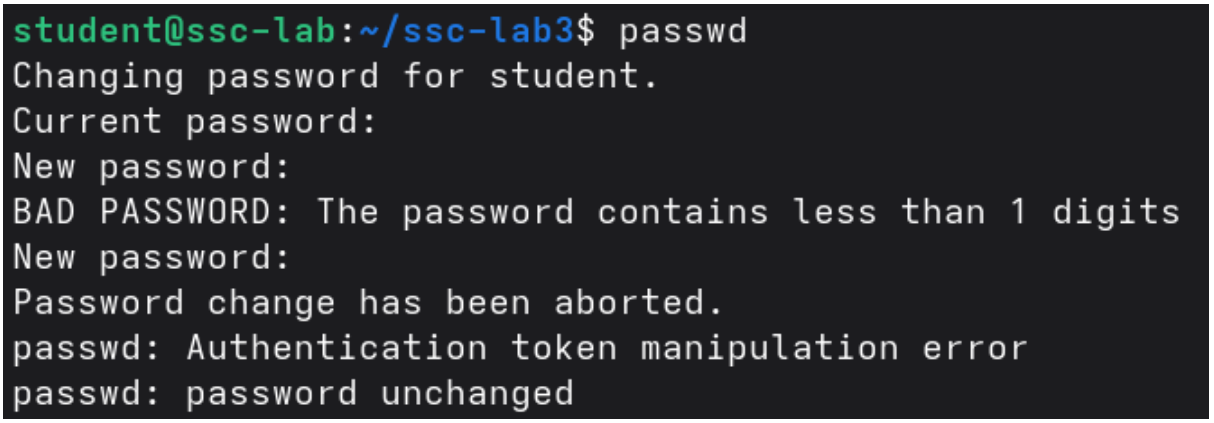


Figure 11 – passwd rejecting the test password with BAD PASSWORD.

2. Lab 4 — OIDC and federated identity

2.1 Preparation questions

P1. The dex discovery document

The issuer, authorization_endpoint and token_endpoint are the following:

"issuer": "http://localhost:5556/dex",

"authorization_endpoint": "http://localhost:5556/dex/auth",

"token_endpoint": "http://localhost:5556/dex/token",

"jwks_uri": "http://localhost:5556/dex/keys"

```
student@ssc-lab:~/ssc-lab4$ curl -s http://localhost:5556/dex/.well-known/openid-configuration \
  | jq '{issuer, authorization_endpoint, token_endpoint, jwks_uri}'
{
  "issuer": "http://localhost:5556/dex",
  "authorization_endpoint": "http://localhost:5556/dex/auth",
  "token_endpoint": "http://localhost:5556/dex/token",
  "jwks_uri": "http://localhost:5556/dex/keys"
}
```

Figure 12 – The dex discovery document.

P2. What the ID token proves

The ID token, signed by the provider, proves to the client that the provider authenticated this user for this client and in this request (the nonce), whereas the access token only serves to access an API, is not addressed to the client and is therefore not proof of identity.

P3. The redirectURIs entry registered for democlient

The redirectURIs entry registered for democlient in dex-config.yaml is:

'http://localhost:3000/callback'.

```
student@ssc-lab:~/ssc-lab4$ cat dex-config.yaml
# Lab 4 (Block 2): dex identity-provider configuration.
# The WHOLE provider is this one file: issuer, one static client, one static
# user. Students are told to read it in full before starting the stack, which
# is the point of choosing dex over a realm import.
#
# SPIKE-CONFIRMED 2026-08-18 on the arm64 test VM against dex v2.41.1: the
# stack serves the discovery document, a full authorization-code + PKCE flow
# yields valid ID/access tokens, strict redirect_uri matching rejects a
# tampered URI, and verify.py/verify_naive.py behave as the lab claims.

issuer: http://localhost:5556/dex

storage:
  type: memory           # keys and state live in memory; fine for a lab, resets on restart

web:
  http: 0.0.0.0:5556

# Skip the consent screen so the flow is observable end to end without a click.
oauth2:
  skipApprovalScreen: true

# --- the relying party (confidential client, also using PKCE) ---------------
staticClients:
  - id: democlient
    name: 'SSC demo client'
    secret: democlientsecret
    redirectURIs:
      - 'http://localhost:3000/callback'
  # NOTE (spike 2026-08-18): a second, public loopback client was tested to see
  # whether dex relaxes redirect-URI matching to accept any port (RFC 8252
  # native-app guidance). dex v2.41.1 does NOT: it exact-matches even public
  # loopback clients (localhost:PORT, 127.0.0.1, [::1] all rejected unless they
  # equal the registered URI). So Part D observes strict validation only; there
  # is no loose case to demonstrate here. The client was removed to keep the
  # provider a single-client, single-user file the students can read in full.

# --- the one login account --------------------------------------------------
# enablePasswordDB + a static password is dex's built-in "local" connector.
# The hash below is bcrypt("password"). Regenerate with:
#   python3 -c 'import bcrypt; print(bcrypt.hashpw(b"password", bcrypt.gensalt(10)).decode())'
enablePasswordDB: true
staticPasswords:
  - email: 'student@ssc.local'
    hash: '$2b$10$2PGStnnva7m1gH38nZKR4.G2Eip9ZeMMEzJpS6KywuOQiMcGtuNra'
    username: 'student'
    userID: '9e1a4c22-0000-4a00-b000-000000000004'
```

Figure 13 – The redirectURIs entry for democlient in dex-config.yaml.

2.2 Part A: The actors and the flow

Exercise 1a

In the Network tab I observed messages 1 to 5 (✔): the requests to /login, /dex/auth and /callback, with redirects 2 and 4 visible as 302/303 responses with a Location header, and the code and the state visible in the /callback URL.

I did not observe messages 6 and 7 (✘): no request to /dex/token appears, because the exchange of the code for tokens is done by client.py, directly with dex.

This matters because the browser is not a trusted channel: whatever passes through it is visible in the history, in the Network tab, in logs or to extensions, and can be diverted if the redirect_uri is poorly validated. That is why only the code travels over the front channel, and it is single-use, short-lived and useless on its own. What actually grants access, the client secret, the code_verifier and the tokens themselves, travels only over the back channel, between the client and dex. So, even if the code is exposed, whoever obtains it cannot exchange it for tokens.

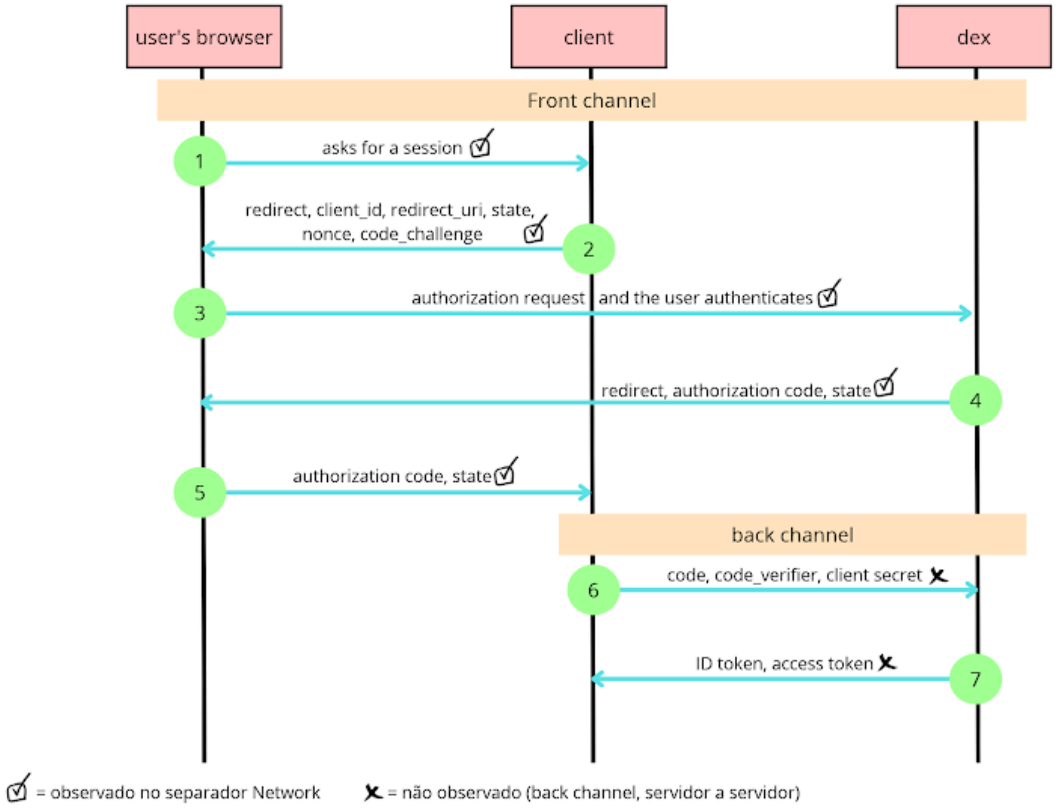


Figure 14 – Annotated sketch of the OIDC flow (Exercise 1a).

2.3 Part B: Run the flow and capture the tokens

Exercise 2a

As the capture shows, the query parameters are:

Parameter	Observed value
client_id	democlient
redirect_uri	http://localhost:3000/callback
response_type	code
scope	openid profile email
state	state-sG05brEIO-Lu
nonce	nonce-s0L7mRDnp-MO
code_challenge	Ux11ZgTC6p9qJsA5MisduUtvwBpOPs1UG4cCDfylSzc
code_challenge_method	S256

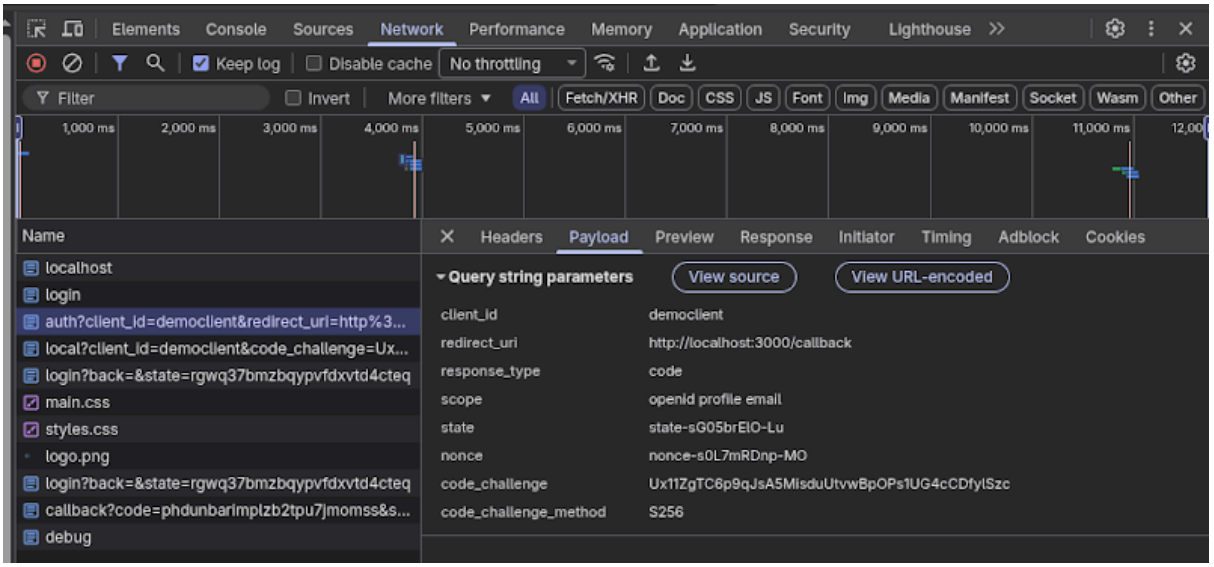


Figure 15 – Query parameters of the request to the authorization endpoint in the Network tab.

Exercise 2b

This matters for 1a because we observe nothing that is on the back channel, so in 1a we record that we only observe what is on the front channel, that is, we only see what communicates with us, because between the client and dex (the provider) we observe nothing, as expected. The capture shows that the code appears in the /callback URL.

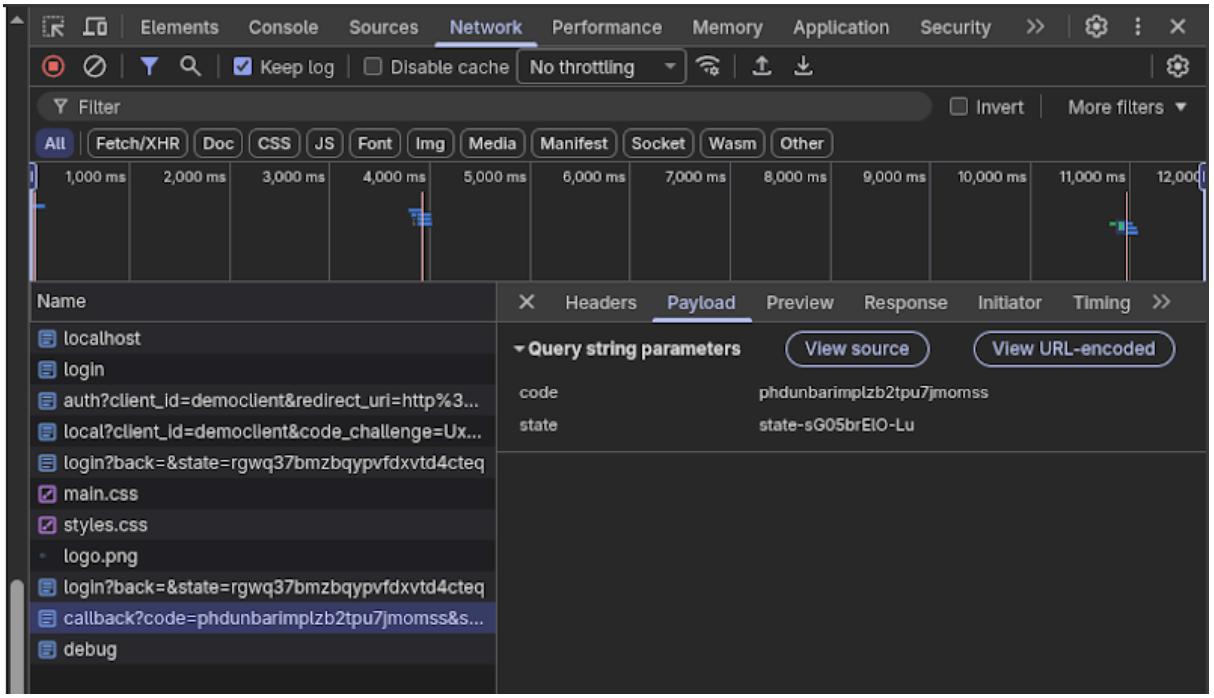


Figure 16 – The redirect back to /callback, with the code and the state in the query string.

The tokens from the client's debug page are the following:

ID token (raw JWT)


```
eyJhbGciOiJSUzI1NiIsImtpZCI6ImQyYzJkYzRiOGNlM2FjMGFhYzhhM2NhOTQ5N2FjMDA0M2Q2M
jlkN2EifQ.eyJpc3MiOiJodHRwOi8vbG9jYWxob3N0OjU1NTYvZGV4Iiwic3ViIjoiQ2lRNVpURmh
OR015TWkwd01EQXdMVFJoTURBdFlqQXdNQzB3TURBd01EQXdNREF3TURRU0JXeHZZMkZzIiwiYXVk
IjoiZGVtb2NsaWVudCIsImV4cCI6MTc5Mjg3ODg2MSwiaWF0IjoxNzkyNzkyNDYxLCJub25jZSI6I
m5vbmNlLXMwTDdtUkRucC1NTyIsImF0X2hhc2giOiJEbzhrbFZmakNVYjBvb2RKSW9oTE9RIiwiY1
9oYXNoIjoibXdaSEJueTJoTGQ5NllBNmt1N0pCUSIsImVtYWlsIjoic3R1ZGVudEBzc2MubG9jYWw
iLCJlbWFpbF92ZXJpZmllZCI6dHJ1ZSwibmFtZSI6InN0dWRlbnQifQ.G8INbFeIxZIXDonluexiS
dlRo8n-G17oYsvY5RKho3CqOK73fAzbGJwEGXFWdLCC224Um1XS0q48_Df307v4t9qFuG9plb1swv
Z3cTTjNp77OsL0V2yFIxang84CG30ozzs83KRDPdJN_b9l-f6vw64LOlFoi9cL_nJbJjGBSvq7Ymx
1QH9H_ozCgnvP3YROYnolSDDYU9-SQRQC3-NOxVVzZO8t7D_dJn_eqo14VZe1eklZhaJjN8gx_j06
eoFPojDX7xKadWNuGzDdufcD_iO1vQmJQ8lQjzs2g0czvbfQxC9Xypz2DkcuP974UqQ6iyrpAXqxw
JEKkI8p9DMIfA
```


ID token claims (decoded, NOT verified)


```
{
  "iss": "http://localhost:5556/dex",
  "sub": "CiQ5ZTFhNGMyMi0wMDAwLTRhMDAtYjAwMC0wMDAwMDAwMDAwMDQSBWxvY2Fs",
  "aud": "democlient",
  "exp": 1792878861,
  "iat": 1792792461,
  "nonce": "nonce-s0L7mRDnp-MO",
  "at_hash": "Do8klVfjCUb0oodJIohLOQ",
  "c_hash": "mwZHBny2hLd96YA6ku7JBQ",
  "email": "student@ssc.local",
  "email_verified": true,
  "name": "student"
}
```


Figure 17 – The raw ID token and its decoded header and payload on the client's debug page.



Figure 18 – The raw access token on the client's debug page.

Exercise 2c

1.

Check	Does client.py do it?	Where it shows
Fetch the public key from the jwks_uri (choosing by kid)	No	it never contacts /dex/keys
Verify the signature	No	decode_payload only decodes
Pin the algorithm (allow-list, reject alg: none)	No	the header is not even read
Verify the iss	No	the constant DEX (http://localhost:5556/dex) exists, but it is never compared with the token's iss
Verify the aud	No	the constant CLIENT (democlient) exists, but it is never compared with the aud
Verify the exp	No	there is no comparison with the current time; an expired token would be accepted
Use the sub (with the iss) as the identity, not the email	No	it shows the email
Compare the nonce with the one it sent	No	it stores the nonce in PENDING in /login, but pend["nonce"] is never used in /callback

2. If an attacker could mint or replay an ID token, the client would accept it, because it checks nothing in it. By minting a token, the attacker chooses the email and the sub and logs in as any user. Since the signature and the iss are

not verified, the attacker does not even need dex's key: their claims are treated as if they came from the provider. Since the exp and the nonce are not verified, an old or stolen token keeps being accepted indefinitely. Since the aud is not verified, a token issued for another application is also accepted.

“We use OIDC” describes the protocol used, not the security obtained. client.py follows the flow correctly (PKCE, state, nonce) and looks correct in the Network tab, but it does not validate the ID token. The protocol only provides the mechanisms; security depends on the client performing the checks. Here the flaw is neither in the protocol nor in the cryptography, but in a validation omission in the client's code.

2.4 Part C: What the tokens prove

Exercise 3a

Claim	Observed value	Security role	If the client ignores this claim, an attacker can…
iss (issuer)	http://localhost:5556/dex	it shows whether the token came from the configured provider and not from any other	use a genuine token from another issuer. The attacker has an issuer they control, and the token from that issuer is correctly signed with that

			issuer's keys. Since the client does not check the iss, it accepts it.
aud (audience)	democlient	it shows whether the token was issued for this client	use a genuine token issued by dex for another application and present it to this client
exp (expiry)	1792878861	it limits how long a stolen token is useful	reuse ID tokens that have already expired
sub (subject)	CiQ5ZTFhNGMyMi0wMDAwLTRhMDAtYjAwMC0wMDAwMDAwMDAwMDQSBWxvY2Fs	stable identifier of the user at this issuer	obtain a genuine token with the victim's email and log in to the victim's account, that is, if the check is made with the email and not

			with the sub
nonce	nonce-s0L7mRDnp-MO	it binds the token to the request this client made	reuse an ID token stolen or issued in another login, injecting it into a flow this client did not start, because the client does not confirm that the nonce is the one it sent



Figure 19 – Decoded header and payload of the ID token.

Exercise 3b

The token header indicates the algorithm (alg: RS256) and the identifier of the key used to sign it (kid). From the configured issuer, the client obtains the jwks_uri (http://localhost:5556/dex/keys) through the discovery document and fetches the JWKS from there, the set of dex's public keys. It chooses the key whose kid is equal to the one in the header and verifies the signature with it.

The token's kid (d2c2dc4b8ce3ac0aac8a3ca9497ac0043d629d7a) matches the second key in the JWKS. The JWKS contains three keys because dex rotates keys: it signs with a new key, but keeps the previous ones published while there are valid tokens signed with them. The kid is what allows the client to choose the right key among them.

```
student@ssc-lab:~/ssc-lab4$ curl -s http://localhost:5556/dex/keys | jq '.keys[] | {kid, alg, kty}'
{
  "kid": "b4a0fcab32f51b8a5f5be028ba30c56557bb0835",
  "alg": "RS256",
  "kty": "RSA"
}
{
  "kid": "d2c2dc4b8ce3ac0aac8a3ca9497ac0043d629d7a",
  "alg": "RS256",
  "kty": "RSA"
}
{
  "kid": "40e29b93d7b230831a35a0523a407f7270338dda",
  "alg": "RS256",
  "kty": "RSA"
}
```

Figure 20 – The dex JWKS, with its three keys identified by kid.

Exercise 3c

The access token is used to authorise calls to an API and is not addressed to the client. Unlike the ID token, the access token does not serve to prove identity, since it does not indicate that the user authenticated for this client in this request (in the ID token we have the aud and the nonce). Therefore, a client that accepts an access token as proof of identity will also accept an access token obtained by another application, for example an application controlled by the attacker, where the attacker obtains a victim's access token through the victim's login; the attacker would then be able to log in to the victim's account.

2.5 Part D: Misconfiguration 1, redirect_uri validation

Exercise 4a

The step that was missing was the login at dex. Since authorisation was not granted to us, because the redirect_uri http://localhost:3000.attacker.example/callback is not registered at dex (the provider) for democlient, dex never redirected to this address and never sent the code to the attacker.

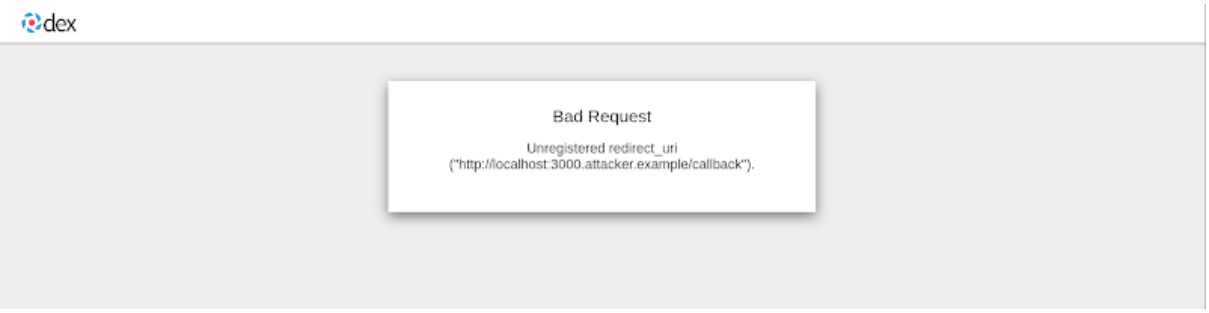


Figure 21 – dex refusing the request with “Unregistered redirect_uri”.

The first response to the authorisation request (/dex/auth?…) has status 302 and Location: /dex/auth/local?client_id=democlient&…&redirect_uri=http%3A%2F%2Flocalhost%3A3000.attacker.example%2Fcallback&…. The second (/dex/auth/local?…) has status 400 Bad Request, with no Location, and its body is the error page “Unregistered redirect_uri (”http://localhost:3000.attacker.example/callback”)”.

The first 302 does not mean that dex accepted the redirect_uri. It is an internal redirect from dex to its login connector (local), which receives the parameters unchanged and without any code. The validation is done in the next step, which rejects the request with 400. If dex had accepted it, there would have been, at the end of the flow, a redirect to http://localhost:3000.attacker.example/callback with a code, which did not happen.

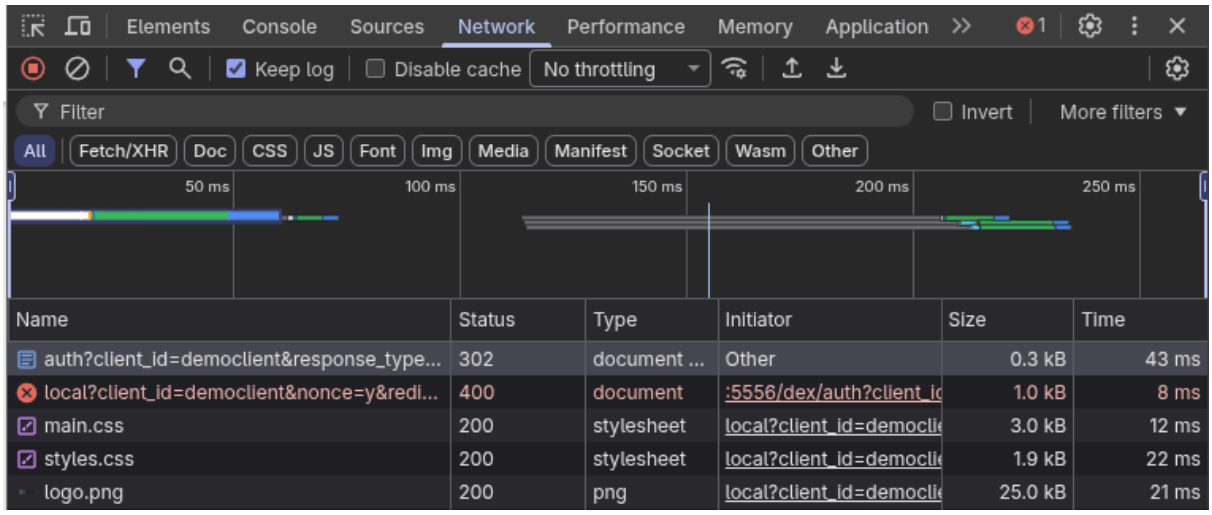


Figure 22 – Network tab: the 302 response from /dex/auth followed by the 400 from /dex/auth/local.

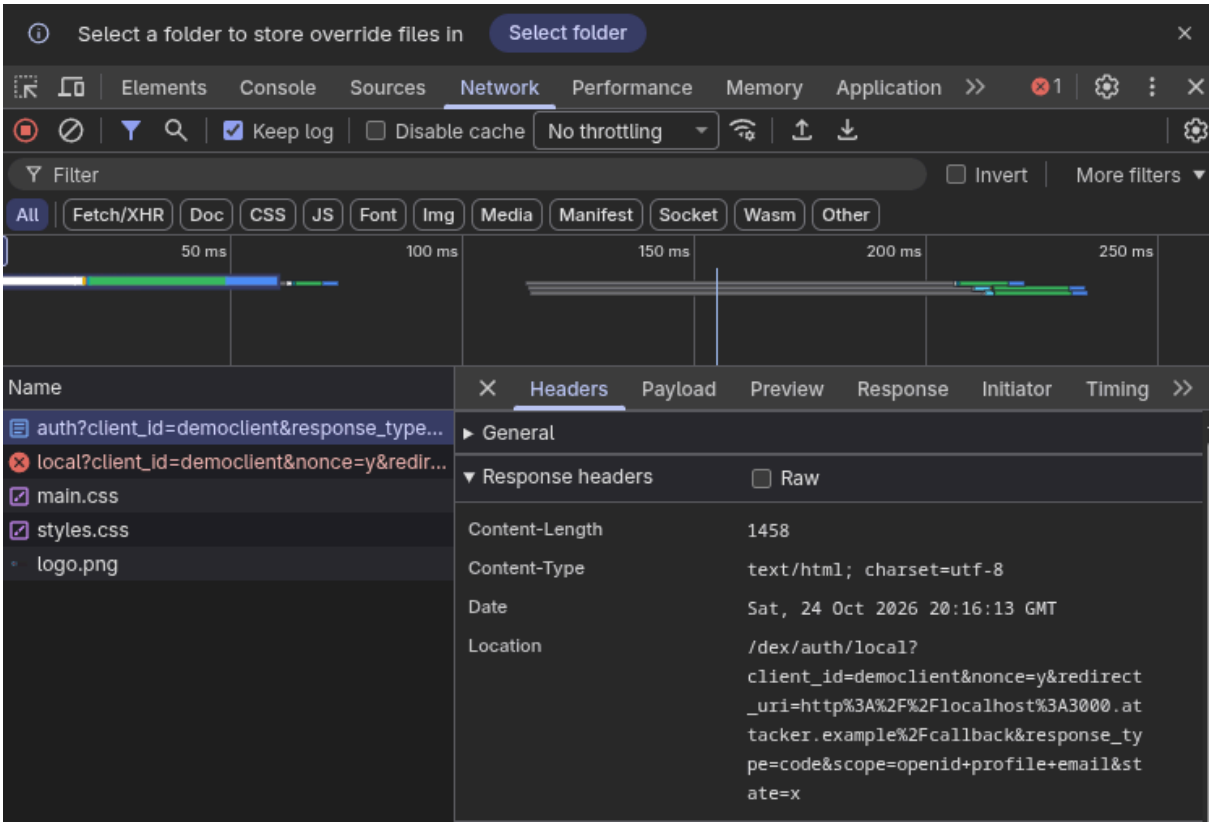


Figure 23 – Response headers of the 302, with the Location pointing to the local connector.

registered in P3: http://localhost:3000/callback

tampered: http://localhost:3000.attacker.example/callback

The two strings match in the first 21 characters (http://localhost:3000). After that, the registered value has /callback, while the tampered one has .attacker.example/callback.

A prefix rule would accept the tampered value. However, dex compares the redirect_uri exactly with the redirectURIs registered for democlient. Since these two strings are not equal, the comparison failed and dex rejected the request with 400 “Unregistered redirect_uri”.

Exercise 4b

The attacker starts a login at the legitimate client and copies the authorisation URL it generates (with a valid state and code_challenge), changing only the redirect_uri to an address of their own that passes the loose rule. They then convince the victim to open that URL. Since the victim already has a session at

the provider, the provider does not ask for the password. Since the redirect_uri comparison rule is careless, the provider accepts it and redirects the victim's browser to the attacker's address, sending the authorization code and the state.

With the code and the state, the attacker opens the client's /callback in their own browser. The state is valid, so the client exchanges the code for tokens on the back channel, using the code_verifier it stored, and creates a session with the victim's identity in the attacker's browser. The attacker thus gains access to the victim's account at that client.

Exercise 4c

i) The attacker has to present the code, the code_verifier and the client secret at the token endpoint. However, the attacker only has the code, so they would also have to present the code_verifier and the client secret, and neither of these two elements passes through the browser, unlike the code. Only the code_challenge passed through the browser, from which the code_verifier cannot be obtained, and the client secret is only on the client's server. Therefore, the provider rejects the exchange, and the leaked code is useless to the attacker.

Note: in client.py, the request to the token endpoint over the back channel includes the code, the redirect_uri, the client_id, the client_secret and the code_verifier; in the answer I only mention the most important ones from a security point of view.

ii) Since it is the attacker who builds the authorization URL and chooses the code_challenge, it is the attacker who generates the code_verifier, and so they know it. Since the client is public, there is no client secret either. Thus, if the victim's code reaches them, the attacker can exchange it directly at the token endpoint (with the client_id, the redirect_uri and their own code_verifier) and receives the victim's tokens. In this case, PKCE only guarantees that the code is exchanged by whoever generated the verifier, who is the attacker themselves: it protects against the theft of a code from someone else's flow, but not against a flow started by the attacker. The only thing left between the

attacker and the tokens is the provider's validation of the redirect_uri: with exact matching, the code is never sent to the attacker's address; with a loose rule, nothing else stops it.

Note: if the attacker instead handed the code to the legitimate client, the client would send its own code_verifier, which does not match the challenge chosen by the attacker, and the provider would reject the exchange.

Exercise 4d

1. The rule https://*.example.com/callback accepts, for example, https://attacker.example.com/callback. For the code to reach the attacker, they have to control a server on a subdomain of example.com. With the wildcard, the rule accepts any subdomain, and not only the registered one (app.example.com).

2. The unanchored substring rule accepts, for example, https://attacker.example/callback?x=app.example.com, because the text app.example.com appears in the URI, even if only in the query. The host is attacker.example, a domain of the attacker. For the attack to work, the attacker only needs a server on a domain of their own.

3. The provider should use exact matching: compare the redirect_uri of the request, character by character, with each of the redirectURIs registered for that client, and reject the request if it is not equal to any of them. The three loose rules accept URIs different from the registered one, as long as they share a part with it (the beginning, in the prefix rule; the end of the domain, in the wildcard; any piece, in the substring). With exact matching, no URI different from the registered one passes, so the code can only be delivered to the addresses that the owner of the client registered.

2.6 Part E: Misconfiguration 2, failing to validate the ID token

Exercise 5a

Note: for this exercise I did a new login, since the previous token was already old.

verify.py rejects the tampered token: [REJECTED: InvalidSignatureError: Signature verification failed]. verify.py reads the kid in the token header, fetches the corresponding public key from dex's jwks_uri and verifies the signature over the token's header and payload. The signature was computed by dex, with its private key, over the original header and payload. Since we changed the sub and the email, the signature no longer matches the payload, and the verification fails. To produce a valid signature for the altered payload, the attacker would need dex's private key. As a control, the original token ($ID) is accepted by verify.py.



Figure 24 – verify.py: original token accepted and tampered token rejected.

Exercise 5b

verify_naive.py accepts the tampered token and shows sub = attacker and email = attacker@evil.example (see Figure 25). The check it skipped, and the one that would have detected this change, is the signature check: it does not fetch dex's public key from the jwks_uri, nor does it verify the signature over the header and the payload. It merely decodes the payload from base64url and trusts what is there. It does not check any claim either (iss, aud, exp, nonce), but in this case those claims had not been changed, and only the signature would have revealed the tampering.

The alg: none attack and “just decode the JWT” are two faces of the same flaw: in both, the signature is never verified against a trusted key. In the first, the attacker writes alg: none in the header, and a verifier that decides based on the header concludes there is nothing to verify. In the second, the verifier does not even try to verify. In both, what the token says is accepted without anyone

having vouched for it. The defence is the same: always verify the signature with the provider's key and pin the accepted algorithms, instead of letting the token choose.

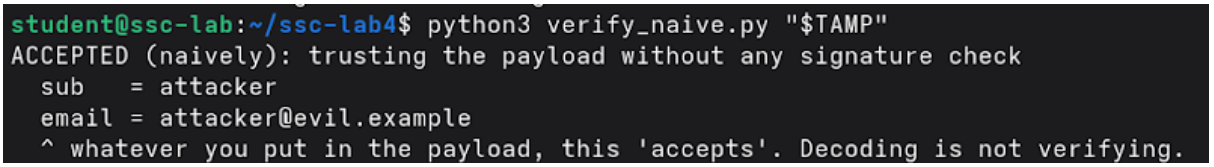


Figure 25 – verify_naive.py accepting the tampered token.

Exercise 5c

Before trusting an ID token, a correct OIDC client has to perform the following validations, in this order:

1. Validate the algorithm in the header, that is, if it is not in the client's list of accepted algorithms, the client rejects this ID token. This prevents alg: none or an algorithm chosen by the attacker.

2. Validate the signature with the kid in the header: the client fetches the corresponding public key from the JWKS of the configured provider (at the jwks_uri obtained through discovery from the issuer), or uses a cached copy. With that key, it verifies the signature over the header and the payload. If the verification fails (the token was altered or was not signed by dex), the client rejects the token.

3. Validate the claims: whether the iss is equal to the configured one, whether the aud contains the client_id, whether the token has not yet expired (the exp has not passed) and whether the nonce is equal to the one the client sent in this login. If any of them fails, the client rejects the token.

If all these validations pass, the user is identified by the pair (iss, sub), and not by the email.

3. Lab 5 — SSH certificates, authorisation and hardening

3.1 Preparation questions

P1. Running containers and generated keys

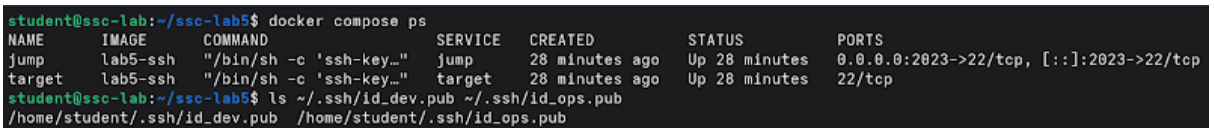


Figure 26 – Output of docker compose ps and listing of the id_dev and id_ops public keys.

P2. Two operational problems of plain public keys

Plain SSH public keys leave two operational problems unsolved: distribution and host authenticity.

The distribution problem is that trust in a key exists only in the local copy on each host, in the authorized_keys file. To give someone access, their public key has to be added on every host, and to take that access away it has to be removed from all of them. Since there is no central revocation nor any way of knowing on which hosts a key still works, a single oversight is enough for someone to keep access they should no longer have, without anyone noticing.

The host-authenticity problem results from SSH using TOFU (Trust On First Use). On the first connection, the client shows the fingerprint of the host key, the user types yes and the public key is stored in known_hosts. That first connection only proves that the other side has a key, not that the key belongs to the right host. If there is a man-in-the-middle attack at that moment, the user may be redirected to the attacker's host without knowing it. The defence would be to compare the fingerprint before accepting it, but in practice almost nobody does so, and the problem repeats every time a host is rebuilt and generates new keys.

Neither of these problems is a flaw in the cryptography, since the private key never leaves the user's computer. They are management problems that get worse with scale: the more users and hosts there are, the more copies there are to maintain and the more fingerprints there are to verify.

3.2 Part A: Key-based authentication and its limits

Exercise 1a

Commands I used to install the dev public key in the dev account, run from the VM as root of the containers.

On target:

```
docker compose exec -T target sh -c 'mkdir -p /home/dev/.ssh \
&& cat > /home/dev/.ssh/authorized_keys \
&& chown -R dev:dev /home/dev/.ssh \
&& chmod 700 /home/dev/.ssh && chmod 600 /home/dev/.ssh/authorized_keys'
< ~/.ssh/id_dev.pub
```

On jump (the same, with jump instead of target):

```
docker compose exec -T jump sh -c 'mkdir -p /home/dev/.ssh \
&& cat > /home/dev/.ssh/authorized_keys \
&& chown -R dev:dev /home/dev/.ssh \
&& chmod 700 /home/dev/.ssh && chmod 600 /home/dev/.ssh/authorized_keys'
< ~/.ssh/id_dev.pub
```

target came to trust my public key because it was placed in the /home/dev/.ssh/authorized_keys file of the dev account. I could not do this over SSH, because I did not have access yet: it was the root of the container, acting as administrator, who copied the key manually, and I had to repeat the same process on jump. From then on, at each login sshd checks whether the public key I present is in that file and asks me for a signature made with the corresponding private key, which never leaves my machine.

This model does not scale because trust exists only in the local copy on each host. With 500 users and 200 hosts there would be 100,000 entries to maintain, and every user who joins or leaves would require changing every host. There is no central revocation nor any way of knowing on which hosts a key still works, so a single oversight is enough for someone to keep access they should no longer have.

On the first connection I had to accept two fingerprints, one for jump and another for target, without any way of verifying them. Accepting them blindly exposes me to a man-in-the-middle attack: if an attacker places themselves in that first connection, I store their key in known_hosts and start trusting their host without knowing it. The defence would be to compare each fingerprint with the real one, obtained through an independent channel, but that would have to be done by every user for every host, and repeated every time a host is rebuilt and generates new keys. In practice nobody does it, and the REMOTE HOST IDENTIFICATION HAS CHANGED! warnings become so frequent that users ignore them, including when they are a real attack.

```
student@ssc-lab:~/ssc-lab5$ ssh -J jump -i ~/.ssh/id_dev dev@target
The authenticity of host '[127.0.0.1]:2023 ([127.0.0.1]:2023)' can't be established.
ED25519 key fingerprint is SHA256:IPkEdr4ettB9ZuIZi06eXfs70UBuR5X3xnObC+7YbhU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[127.0.0.1]:2023' (ED25519) to the list of known hosts.
The authenticity of host 'target (<no hostip for proxy command>)' can't be established.
ED25519 key fingerprint is SHA256:W7rWQ+sp88Eahj5Zgll96CuDaTRNxRv82/yk7AvliyI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'target' (ED25519) to the list of known hosts.
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

a5ab2bb220ae:~$ ls
a5ab2bb220ae:~$ whoami
dev
a5ab2bb220ae:~$ hostname
a5ab2bb220ae
```

Figure 27 – First connection to jump and target, accepting the fingerprints.

3.3 Part B: An SSH certificate authority: authentication, authorisation, revocation

Exercise 2a — User certificate

Command I used in the pre-lab to create the user CA, whose private key signs the dev and ops certificates:

ssh-keygen -t ed25519 -f ~/.ssh/user_ca -N '' -C 'SSC user CA'

The certificate has the principal dev, serial 1, and is valid from 2026-10-30T21:46:00 to 2026-10-30T22:47:24. The end corresponds to one hour after issuance; the start was set slightly before that moment, to tolerate clock differences between machines.

Certificates solve the first problem, distribution: each host trusts only the CA's public key (TrustedUserCAKeys), so granting or renewing access becomes a matter of issuing a certificate, without changing any host.

Compared with a permanent entry in authorized_keys, which works until someone remembers to remove it, a short validity interval limits exposure in time: access expires on its own, without removing anything on any host, and if the key and the certificate are stolen, the attacker can only use them until the end of the validity. Renewing access is also simple, because the CA just issues a new certificate, whereas rotating a distributed key would require changing it on every host.

However, short validity does not revoke the certificate: if I learn that it was stolen, I cannot invalidate it before the end of the interval, and during that time the attacker uses it as if they were me. For that, a KRL is needed. Validity also does not decide which accounts the certificate allows logging in to, which is an authorisation question handled by the principals and the host's policy, and it does not end sessions that are already open when the certificate expires.

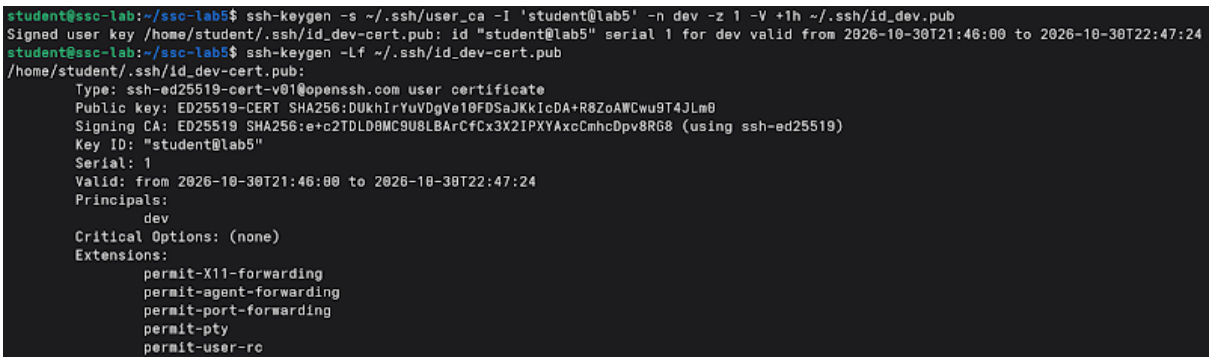


Figure 28 – Issuing and inspecting the dev certificate with ssh-keygen -L.

Exercise 3a — Principals and accounts



Figure 29 – TrustedUserCAKeys configuration on target.



Figure 30 – Logging in as dev with the certificate, and the target log.

When trying to log in as ops with the dev certificate, the client asks for a password. sshd rejects the certificate for that account and, since password authentication is still enabled, the client moves on to the next method.

According to sshd_config(5), when AuthorizedPrincipalsFile is not set (the default is none), the certificate is only accepted if the name of the requested account appears in the certificate's list of principals. After verifying the certificate's signature with the CA public key in TrustedUserCAKeys, its validity and the possession of the private key, sshd compares the account name with the principals. Since the certificate only has the principal dev, the login as dev is accepted and the login as ops is rejected.

At this point, the certificate only allows logging in to the dev account. This authorisation is implicit: it results from the principal name matching the account name, and not from a policy written by the administrator of target.

```
student@ssc-lab:~/ssc-lab5$ ssh -J jump -i ~/.ssh/id_dev ops@target
ops@target's password:
```

Figure 31 – Attempt to log in as ops with the dev certificate.

With the log terminal running, target recorded “Certificate invalid: name is not a listed principal”, which confirms the prediction: the certificate was authenticated, but rejected for the ops account because ops is not among its principals. The client, not knowing the reason, moved on to the next method and asked for a password.

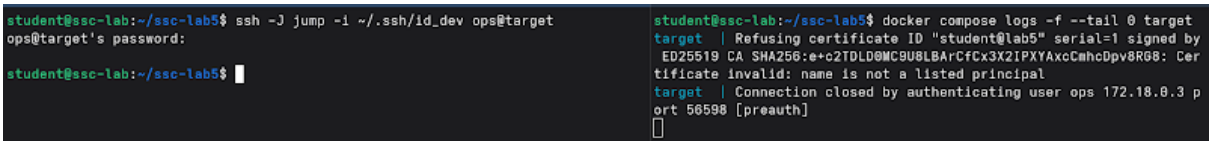


Figure 32 – target log: name is not a listed principal.

Exercise 4a — Authorisation model by principals

```
student@ssc-lab:~/ssc-lab5$ docker compose exec target sh
/ # ls /etc/ssh/auth_principals
dev  ops
/ # cat /etc/ssh/auth_principals/dev
dev
/ # cat /etc/ssh/auth_principals/ops
ops
```

Figure 33 – Principals files in /etc/ssh/auth_principals.

```
student@ssc-lab:~/ssc-lab5$ docker compose exec target sh
/ # cat /etc/ssh/sshd_config
# Lab 5: the sshd configuration of `jump` and `target`, as you receive it.
#
# This is the starting point of the lab. What is missing is missing on
# purpose, because adding it is the lab:
#   TrustedUserCAKeys          Exercise 3
#   AuthorizedPrincipalsFile   Exercise 4
#   RevokedKeys                Exercise 5
#   the hardening directives   Exercise 6
#
# docker-compose.yml mounts your user CA's public key at
# /etc/ssh/user_ca.pub. Nothing uses it until you write the
# TrustedUserCAKeys line in Exercise 3.
#
# After editing this file inside a container, check it with `sshd -t`
# and reload the daemon with `kill -HUP 1`: sshd is PID 1 there, and
# there is no systemd. `sshd -T` prints the effective configuration,
# one keyword per line.
#

Port 22

# These are the defaults, written out so that this file and what `sshd -T`
# reports agree before you change anything.
PasswordAuthentication yes
PubkeyAuthentication yes
PermitRootLogin prohibit-password
KbdInteractiveAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys

# Needed by Part D (the agent-forwarding hijack, Exercises 7 and 8). Not part of
# the hardening exercises: leave it.
AllowAgentForwarding yes
TrustedUserCAKeys /etc/ssh/user_ca.pub
/ # echo 'AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u' >> /etc/ssh/sshd_config
/ # tail -3 etc/ssh/sshd_config
AllowAgentForwarding yes
TrustedUserCAKeys /etc/ssh/user_ca.pub
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u
/ # sshd -t
/ # kill -HUP 1
/ # exit
```

Figure 34 – sshd_config of target and the AuthorizedPrincipalsFile setting.

With the dev certificate, the attempt to log in as ops resulted in a password prompt and the attempt to log in as dev was accepted. In the log, the first was recorded as “Certificate does not contain an authorized principal” and the second as “Accepted publickey for dev ... ID student@lab5 (serial 1)”, which identifies the certificate used by its Key ID and serial.

The two attempts presented exactly the same credential: the same certificate, signed by the same CA, within its validity and with the same private key. Authentication was the same in both cases; the difference lies in authorisation.

With AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u, sshd replaces %u with the requested account and checks whether any of the certificate's principals is listed in that file. For dev@target, it opens auth_principals/dev, which lists dev, and the login is accepted. For ops@target, it opens auth_principals/ops, which lists only ops; the principal dev is not there, so the certificate is rejected and the client moves on to the next method, asking for a password.

The result is the same as in 3a, but the reason in the log changed (“name is not a listed principal” gave way to “does not contain an authorized principal”): the decision is now made by a policy written by the administrator of target, and not by the match between the principal name and the account name.



Figure 35 – Attempts as ops and as dev with AuthorizedPrincipalsFile, and the target log.

Exercise 4b — ops certificate: authentication and authorisation

Command I used to sign the ops certificate with the user CA (Key ID student-ops, principal ops, serial 2, one-hour validity):

ssh-keygen -s ~/.ssh/user_ca -I 'student-ops' -n ops -z 2 -V +1h ~/.ssh/id_ops.pub

With the ops certificate, the login to the ops account was accepted (log: “Accepted publickey for ops ... ID student-ops (serial 2)”).

The CA decided, in advance and outside target, that whoever holds id_ops has the principal ops during the validity interval, without knowing anything about the host's accounts.

AuthorizedPrincipalsFile decided, on target and at login time, that the principal ops may log in to the ops account, because it is listed in /etc/ssh/auth_principals/ops, a file controlled by the host's administrator.

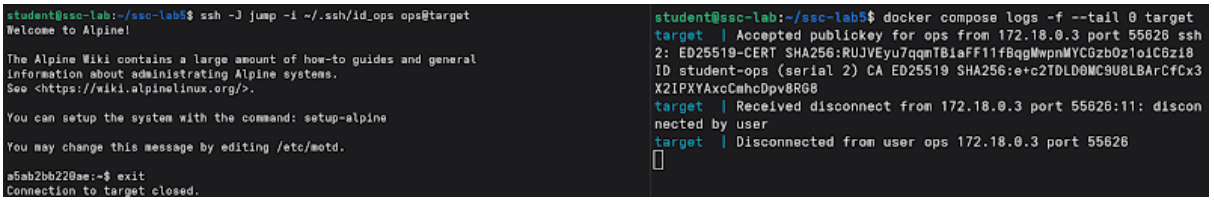


Figure 36 – Logging in as ops with the ops certificate.

Note: through an oversight, I did not record the inspection of the ops certificate during the lab. I therefore ran the command ssh-keygen -Lf ~/.ssh/id_ops-cert.pub afterwards, and its output is shown in the following figure.

```
student@ssc-lab:~/ssc-lab5$ ssh-keygen -Lf ~/.ssh/id_ops-cert.pub
/home/student/.ssh/id_ops-cert.pub:
        Type: ssh-ed25519-cert-v01@openssh.com user certificate
        Public key: ED25519-CERT SHA256:RUJVEyu7qqmTBiaFF11fBqgMwpnMYCGzbOz1oiC6zi8
        Signing CA: ED25519 SHA256:e+c2TDLD0MC9U8LBArCfCx3X2IPXYAxcCmhcDpv8RG8 (using ssh-ed25519)
        Key ID: "student-ops"
        Serial: 2
        Valid: from 2026-11-01T01:31:00 to 2026-11-01T02:32:19
        Principals:
                ops
        Critical Options: (none)
        Extensions:
                permit-X11-forwarding
                permit-agent-forwarding
                permit-port-forwarding
                permit-pty
                permit-user-rc
```

Figure 37 – Inspection of the ops certificate with ssh-keygen -L.

Exercise 5a — Revocation with a KRL

```
student@ssc-lab:~/ssc-lab5$ ssh-keygen -k -f ~/revoked_keys ~/.ssh/id_ops-cert.pub
Revoking from /home/student/.ssh/id_ops-cert.pub
student@ssc-lab:~/ssc-lab5$ ssh-keygen -Qf ~/revoked_keys ~/.ssh/id_ops-cert.pub
ssh-keygen -Qf ~/revoked_keys ~/.ssh/id_dev-cert.pub
/home/student/.ssh/id_ops-cert.pub (lab5 ops identity): REVOKED
/home/student/.ssh/id_dev-cert.pub (lab5 dev identity): ok
student@ssc-lab:~/ssc-lab5$ docker cp ~/revoked_keys target:/etc/ssh/revoked_keys
Successfully copied 2.05kB to target:/etc/ssh/revoked_keys
```

Figure 38 – Creating and checking the KRL and copying it to target.

```
student@ssc-lab:~/ssc-lab5$ docker compose exec target sh
/ # ls -l /etc/ssh/revoked_keys
-rw-r--r--    1 ops      dev           121 Oct 31 19:11 /etc/ssh/revoked_keys
/ # echo 'RevokedKeys /etc/ssh/revoked_keys' >> /etc/ssh/sshd_config
/ # tail -3 /etc/ssh/sshd_config
TrustedUserCAKeys /etc/ssh/user_ca.pub
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u
RevokedKeys /etc/ssh/revoked_keys
/ # sshd -t
/ # kill -HUP 1
/ # exit
```

Figure 39 – RevokedKeys configuration on target.

Before revocation, the ops certificate allowed logging in to the ops account (see Figure 36). After creating the KRL, ssh-keygen -Qf reported the ops certificate as revoked and the dev certificate as not revoked.

With the KRL active on target, the attempt with the ops certificate resulted in a password prompt, and the log recorded “Authentication key ED25519-CERT SHA256:RUJV... revoked by file /etc/ssh/revoked_keys”. The certificate was still within its validity, signed by the CA and with a principal authorised for the ops account, as seen in Exercise 4; here it was rejected only because it is in the KRL. The log message is different from the previous ones because the rejection happens in the authentication phase, before the principals policy is consulted. Since the client does not know the reason, it moved on to the next method and asked for a password.

The attempt with the dev certificate was accepted, and the log recorded “Accepted publickey for dev ... ED25519-CERT ... ID student@lab5 (serial 1)”. Revocation is selective: the KRL contains only the ops certificate, so the dev certificate, from another key and with another serial, keeps working normally.

```
student@ssc-lab:~/ssc-lab5$ ssh -J jump -i ~/.ssh/id_ops ops@target
ops@target's password:

student@ssc-lab:~/ssc-lab5$ ssh -J jump -i ~/.ssh/id_dev dev@target
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

a5ab2bb220ae:~$ exit
Connection to target closed.
```

```
student@ssc-lab:~/ssc-lab5$ docker compose logs -f --tail 0 target
target  | Authentication key ED25519-CERT SHA256:RUJVEyu7qqmTBiaFF1
1fBqgMwpnMYCGzbOz1oiC6zi8 revoked by file /etc/ssh/revoked_keys
target  | Connection closed by authenticating user ops 172.18.0.3 p
ort 52616 [preauth]
target  | Accepted publickey for dev from 172.18.0.3 port 59318 ssh
2: ED25519-CERT SHA256:DUkhIrYuVDgVe10FDSaJKkIcDA+R8ZoAWCwu9T4JLm0
ID student@lab5 (serial 1) CA ED25519 SHA256:e+c2TDLD0MC9U8LBArCfCx
3X2IPXYAxcCmhcDpv8RG8
target  | Received disconnect from 172.18.0.3 port 59318:11: discon
nected by user
target  | Disconnected from user dev 172.18.0.3 port 59318
```

Figure 40 – Attempts with the KRL active, and the target log.

Exercise 5b — The four-way distinction

CA signature: establishes, in advance and outside the host, that whoever holds the key has the principals stated in the certificate; it does not limit where or for how long, it only guarantees the identity on which the other mechanisms act.

AuthorizedPrincipalsFile: limits, on each host and for each account, which principals may log in to it; this is a local decision, made at login by the host's administrator, independent of the CA.

Validity interval: limits how long the certificate is accepted, automatically and fixed at issuance, with no possibility of being shortened later.

KRL / RevokedKeys: ends a specific certificate before the end of its validity, when the administrator decides, only on the hosts that load the KRL.

With short validities alone, the effect of 5a cannot be obtained, because validity is fixed at issuance: if the certificate and the key are stolen, the attacker can use them freely until the end of the interval, and noticing the theft does not shorten that time. In the 5a test, the ops certificate was still within its validity, so only the KRL could reject it. Validity limits exposure; only revocation ends it.

3.4 Part C: Hardening sshd

Exercise 6a — Effective configuration, counts and the password test

```
student@ssc-lab:~/ssc-lab5$ docker compose exec target sh
/ # echo 'PasswordAuthentication no' >> /etc/ssh/sshd_config
/ # echo 'PermitRootLogin no' >> /etc/ssh/sshd_config
/ # echo 'PubkeyAuthentication yes' >> /etc/ssh/sshd_config
/ # echo 'KbdInteractiveAuthentication no' >> /etc/ssh/sshd_config
/ # echo 'AuthenticationMethods publickey' >> /etc/ssh/sshd_config
/ # tail -5 /etc/ssh/sshd_config
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
KbdInteractiveAuthentication no
AuthenticationMethods publickey
/ # sshd -t
/ # kill -HUP 1
/ # exit
```

Figure 41 – Hardening directives appended to sshd_config.

```
student@ssc-lab:~/ssc-lab5$ ssh -o PreferredAuthentications=password -J jump dev@target
dev@target: Permission denied (publickey).
student@ssc-lab:~/ssc-lab5$ ssh -J jump -i ~/.ssh/id_dev dev@target
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

a5ab2bb220ae:~$ exit
Connection to target closed.
```

Figure 42 – Password test and certificate login.

The two checks do not agree. The login tests suggest that the hardening was applied, because the password attempt failed and the certificate kept working, but sshd -T shows that only two of the five intended settings are in effect: PubkeyAuthentication yes and AuthenticationMethods publickey. PasswordAuthentication, KbdInteractiveAuthentication and PermitRootLogin still have values different from the ones I wrote (yes, yes and prohibit-password).

Intended settings in effect: 2.

Lines I wrote that the daemon obeys: 1.

Lines I wrote that are inert: 4.

```
student@ssc-lab:~/ssc-lab5$ docker compose exec -T target sshd -T | grep -iE \
  -e '^passwordauthentication|^permitrootlogin|^kbdinteractiveauthentication' \
  -e '^pubkeyauthentication|^authenticationmethods'
permitrootlogin prohibit-password
pubkeyauthentication yes
passwordauthentication yes
kbdinteractiveauthentication yes
authenticationmethods publickey
```

Figure 43 – Output of sshd -T before the fix.

Reading the sshd_config of target from the beginning, I found that the file already contains, before my lines, a block with the default values written out explicitly: PasswordAuthentication yes, PubkeyAuthentication yes, PermitRootLogin prohibit-password and KbdInteractiveAuthentication yes. According to sshd_config(5), for each keyword the first obtained value is used, so my four corresponding lines, appended at the end, are ignored. Three of

them produce a value different from the intended one; the PubkeyAuthentication line is equally inert, although the value in effect matches what I wanted, because it is the original line that the daemon obeys. Only AuthenticationMethods publickey had no earlier line and is obeyed.

So, of the five intended settings, two are in effect; of the five lines I wrote, only one is obeyed by the daemon and four are inert. sshd -t did not detect the problem because a repeated keyword is not a syntax error. To fix it, I removed the four lines of the default block, reloaded sshd, and sshd -T then reported the five intended values.

```
Port 22

# These are the defaults, written out so that this file and what `sshd -T`
# reports agree before you change anything.
PasswordAuthentication yes
PubkeyAuthentication yes
PermitRootLogin prohibit-password
KbdInteractiveAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys

# Needed by Part D (the agent-forwarding hijack, Exercises 7 and 8). Not part of
# the hardening exercises: leave it.
AllowAgentForwarding yes
TrustedUserCAKeys /etc/ssh/user_ca.pub
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u
RevokedKeys /etc/ssh/revoked_keys
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
KbdInteractiveAuthentication no
AuthenticationMethods publickey
```

Figure 44 – End of the sshd_config of target, with the block of default values.

```
/ # sed -i '/^PasswordAuthentication yes$/d; /^PermitRootLogin prohibit-password$/d; /^KbdInteractiveAuthentication yes$/d; /^PubkeyAuthentication yes$/d' /etc
/ssh/sshd_config
/ # echo 'PubkeyAuthentication yes' >> /etc/ssh/sshd_config
/ # grep -nE '^(PasswordAuthentication|PubkeyAuthentication|PermitRootLogin|KbdInteractiveAuthentication|AuthenticationMethods)' /etc/ssh/sshd_config
32:PasswordAuthentication no
33:PermitRootLogin no
34:KbdInteractiveAuthentication no
35:AuthenticationMethods publickey
36:PubkeyAuthentication yes
/ # sshd -t
/ # kill -HUP 1
/ # exit
student@ssc-lab:~/ssc-lab5$ docker compose exec -T target sshd -T | grep -iE \
  -e '^passwordauthentication|^permitrootlogin|^kbdinteractiveauthentication' \
  -e '^pubkeyauthentication|^authenticationmethods'
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
kbdinteractiveauthentication no
authenticationmethods publickey
```

Figure 45 – Fixing sshd_config and the output of sshd -T after the fix.

As for the check made at the start, the password attempt failed with “Permission denied (publickey)”, and the list in parentheses shows that the server only offered the publickey method. Before the fix, the only directive in effect that restricted the methods was AuthenticationMethods publickey, the only one of my lines that the daemon obeyed. On its own, it requires authentication to be done with a key or a certificate, and that is why it removed password and keyboard-interactive from the list, even though PasswordAuthentication and KbdInteractiveAuthentication were still set to yes.

Therefore, passing that test did not prove that the rest of the baseline was applied: the result was the same before and after the fix, because it depended only on AuthenticationMethods. The test said nothing about the other four directives, and it did not even try a root login, so it could not detect that PermitRootLogin was still prohibit-password. If AuthenticationMethods were one day removed, password authentication would become available again. Only sshd -T shows the effective configuration of each directive.

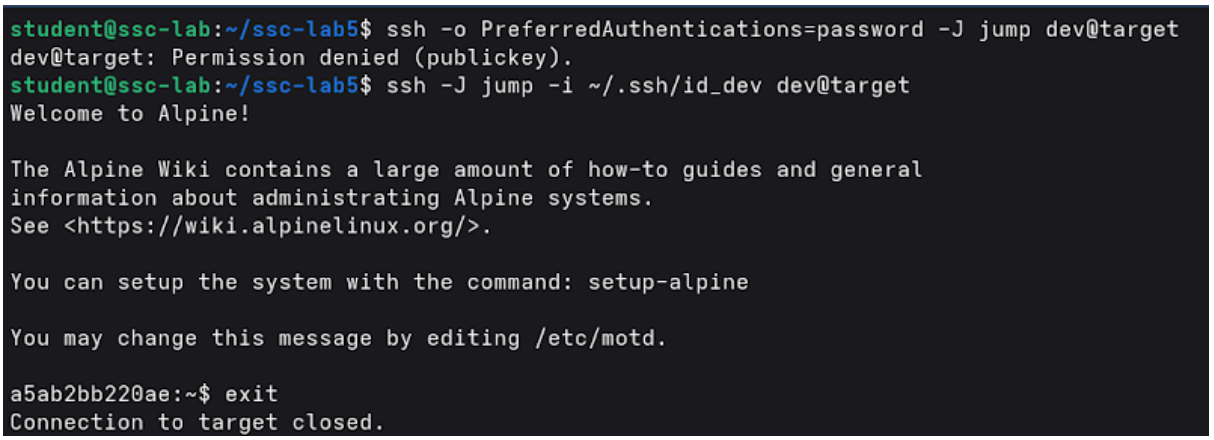


Figure 46 – Password test and certificate login after the fix.

The final sshd_config of target, obtained with docker compose exec target cat /etc/ssh/sshd_config, is the following:

```
student@ssc-lab:~/ssc-lab5$ docker compose exec target cat /etc/ssh/sshd_config
# Lab 5: the sshd configuration of `jump` and `target`, as you receive it.
#
# This is the starting point of the lab. What is missing is missing on
# purpose, because adding it is the lab:
#   TrustedUserCAKeys          Exercise 3
#   AuthorizedPrincipalsFile   Exercise 4
#   RevokedKeys                Exercise 5
#   the hardening directives   Exercise 6
#
# docker-compose.yml mounts your user CA's public key at
# /etc/ssh/user_ca.pub. Nothing uses it until you write the
# TrustedUserCAKeys line in Exercise 3.
#
# After editing this file inside a container, check it with `sshd -t`
# and reload the daemon with `kill -HUP 1`: sshd is PID 1 there, and
# there is no systemd. `sshd -T` prints the effective configuration,
# one keyword per line.
#

Port 22

# These are the defaults, written out so that this file and what `sshd -T`
# reports agree before you change anything.
AuthorizedKeysFile .ssh/authorized_keys

# Needed by Part D (the agent-forwarding hijack, Exercises 7 and 8). Not part of
# the hardening exercises: leave it.
AllowAgentForwarding yes
TrustedUserCAKeys /etc/ssh/user_ca.pub
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u
RevokedKeys /etc/ssh/revoked_keys
PasswordAuthentication no
PermitRootLogin no
KbdInteractiveAuthentication no
AuthenticationMethods publickey
PubkeyAuthentication yes
```

Figure 47 – Final sshd_config of target.

Exercise 6b — Ranking the directives

The three directives that most reduce the remote attack surface of this host are AuthenticationMethods publickey, PasswordAuthentication no and PermitRootLogin no.

The directive “AuthenticationMethods publickey” addresses the threat of password guessing, whether by brute force, by lists of common passwords or by reusing passwords stolen from other services, because it requires every login to be done with a key or a certificate, regardless of whether other methods are enabled. The directive “PasswordAuthentication no” addresses the same

threat directly, by disabling the password method, and works as defence in depth in case AuthenticationMethods is changed or removed. Finally, the directive “PermitRootLogin no” addresses direct remote access to the most privileged account on the system: a compromised credential that gave access to root would give the attacker full control of the host in a single step, without the need to escalate privileges, and on a shared account whose logs do not identify who logged in.

If I could only make one change, I would apply AuthenticationMethods publickey. As seen in 6a, it was the only one of my lines that had an effect before the fix and, on its own, it removed the password and keyboard-interactive methods from the list of methods offered by the server, even though PasswordAuthentication and KbdInteractiveAuthentication were still set to yes. With a single line, it eliminates the whole password-guessing surface, leaving possession of a private key that never leaves the user's computer as the only way in.

Exercise 6c — PasswordAuthentication enabled

With PasswordAuthentication enabled and no other directive requiring a key, such as AuthenticationMethods publickey, an attacker without any credential can try to log in remotely just by guessing or reusing an account's password, something they can do from a distance and automatically, whereas the hardened configuration required possession of a private key, which never leaves the user's computer, and of a valid certificate issued by the CA.

3.5 Part D: The agent-forwarding hijack

Exercise 7a — The attack

The attacker started by looking for dev's private key and certificate on jump, without success: find returned nothing, because with agent forwarding the key is never copied to the remote host. Next, they found the socket of the forwarded agent, which the sshd on jump creates and which forwards requests to the agent running on my VM. They pointed their own ssh at that socket with SSH_AUTH_SOCK=$SOCK: ssh-add -L listed dev's public key and certificate,

confirming access to the agent, and ssh dev@target authenticated them on target as dev. When target asked for a signature, the request travelled through the socket to the agent on the VM, which signed it without knowing that it came from the attacker; target received a valid certificate and a valid signature and let them in.

Keeping the private key secret did not protect anything because the attacker never needed it. What they used was the socket, which gives access to the ability to use the key, and not to the key itself. The agent answers any request that reaches it through the socket, without distinguishing who makes it, so, while the ssh -A session was open, any root on jump could impersonate the victim, even though the key never left the victim's machine.

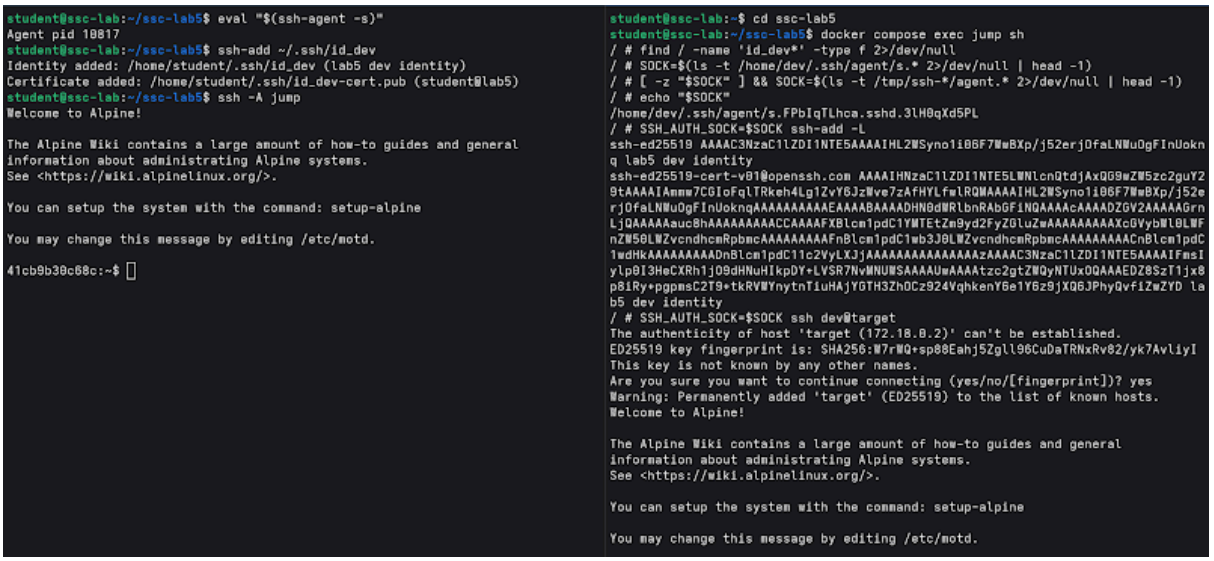


Figure 48 – Hijacking the forwarded agent on jump.

Exercise 8a — Mitigation

ProxyJump is not vulnerable to the hijack of Part D because it is my VM that speaks SSH to target. jump only forwards the encrypted bytes and, since -J does not forward the agent, the sshd on jump does not create any agent socket. The signature is made on my VM, at the request of the VM itself, and a root on jump has nothing it can use. With agent forwarding, on the contrary, it is jump that speaks to target, and to authenticate it needs to ask the agent for the signature; that request travels through a socket that jump itself maintains and

that forwards it to the agent on my VM, which signs without knowing who asked, allowing an attacker on jump to impersonate me.

The rule I would give an operator is: the default must be agent forwarding disabled, and to hop through a bastion one should always use -J (ProxyJump), which offers the same chaining without exposing the agent. -A is only acceptable for a host one trusts completely, since it hands that host the ability to use the agent for the whole session, and only when there is no way to obtain the same result with -J.

For the rare cases in which -A really is necessary, as defence in depth, the key should be loaded with ssh-add -c, which makes every use of the agent ask the user for confirmation on the machine where the agent runs. An attacker on jump would have to wait for a confirmation that would appear on my VM and that I would not authorise, since I had not made the request. In addition, ForwardAgent no should be set per host in ~/.ssh/config, ensuring that the agent is never forwarded to hosts where it should not be, even if the user forgets to disable it on the command line.

```
student@ssc-lab:~/ssc-lab5$ ssh -J jump -i ~/.ssh/id_dev dev@target
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

a5ab2bb220ae:~$ whoami
dev
a5ab2bb220ae:~$ exit
Connection to target closed.
```

Figure 49 – Reaching target with ProxyJump (-J).

Appendix — Reflections

A.1 Lab 3 reflection

Question 1: The main advantage of this architecture is that, since services compose modules instead of having password verification in their own code, the administrator changes the policy in a single place: by changing common-auth, every service that includes it is changed. Moreover, the service knows that we authenticated, but not how: sudo only receives yes or no, so swapping the module, for example to require another authentication method, does not affect it.

However, as we saw in Part C, a single wrong line is enough to lose access to root: when we added a module that did not exist (Module is unknown), the stack failed closed, leaving sudo unavailable, and this is our only path to root. If the error were in common-auth, every service would become unavailable.

This architecture is worth the risk, as long as precautions are taken: keeping a terminal open as root, having a snapshot of the machine and keeping copies of the configuration files before changing them.

Question 2: The account lockout control (pam_faillock) prevents an attacker from trying to guess a user's password in rapid succession, making the attack slow, since they could only make about 90 attempts per hour. However, this has a trade-off: if the attacker does this constantly, they will lock the user's account, leading to a denial of service (DoS). This makes the user enter the correct password and see Sorry, try again, because the account is locked, without knowing it. To cause this DoS, the attacker only needs to know the user's name.

I would recommend that the operator keep a moderate number of attempts, together with strong passwords, and use public keys for remote access. However, this means more things to configure and manage.

A.2 Lab 4 reflection

Question 1: This centralisation brings several advantages: the client never sees the password, authentication is done in a single place, an account is revoked in a single place, and there are fewer login pages and less password reuse.

If the provider is compromised, the attacker can steal the provider's private signing key. With it, they can create any ID token, which will be accepted by every application that trusts this provider. The attacker does not need the passwords, since the tokens they forge are indistinguishable from the real ones: they have the right signature and the right iss and aud, and they would pass verification. Until the compromised key is removed from the JWKS, clients will keep accepting the attacker's tokens, to which the attacker can give any validity they want.

This centralisation is worth it, since it is more convenient and trades many weak points spread around for a single point that can be seriously protected, as long as the provider and its private key are treated as the most critical point of the system. As 5a showed, it is this key that decides what is accepted: only the signature stopped the forged token from passing.

Question 2: Even with verify.py, the attacker can start a login at the client, authenticate at dex with their own account and stop before /callback, keeping a valid code and a state that remains in PENDING. If the victim opens that /callback?code=…&state=…, the client accepts the state, because it does not bind it to any browser, and exchanges the code. verify.py accepts the ID token, because it is genuine: it was issued by dex for democlient, and the nonce matches the one stored in PENDING. The victim ends up logged in to the attacker's account, and the data the victim enters there is within the attacker's reach.

The token correctly says who authenticated, but not who is at the browser. Validating the token is not enough to bind the login to the browser that started it: for that, the state has to be associated with that browser's session (for example, in a cookie) and compared at /callback. A secure login requires validating both the token and the flow.

Note: fixing the state storage in client.py

In client.py, the state is stored in a global dictionary (PENDING), with no link to the browser that started the login. At /callback, the client only checks whether the state exists in PENDING, and not whether it was issued to the browser presenting it. The fix consists of binding the state to the browser's session through a cookie created at /login.

Code change (sketch)

```
# /login
pre = secrets.token_urlsafe(16)
PENDING[pre] = {"state": state, "verifier": verifier, "nonce": nonce}
self._send(302, "", [("Location", f"{DEX}/auth?{params}"),
("Set-Cookie", f"pre={pre}; Path=/; HttpOnly")])

# /callback
pre = <cookie "pre" sent by the browser>
pend = PENDING.pop(pre, None)
if not pend or pend["state"] != state:
<reject the request>
```

Instead of checking whether the state exists, the client checks whether it is the state it issued to this browser.

Normal flow (the victim logs in)

1. The victim's browser requests /login.

2. The client generates state-V, replies with a redirect to dex and sends the cookie pre=V1. It stores PENDING[V1] = state-V.

3. The victim authenticates at dex, which redirects them to /callback?code=…&state=state-V.

4. The browser requests /callback?code=…&state=state-V and automatically sends the cookie pre=V1.

5. The client finds state-V in PENDING[V1], equal to the one in the URL, and accepts.

6. The client exchanges the code for tokens, validates the ID token with verify.py and creates the session.

The attack from question 2, with the fix

1. The attacker requests /login in their browser. The client generates state-A, sends the cookie pre=A1 to the attacker's browser and stores PENDING[A1] = state-A.

2. The attacker authenticates at dex with their own account and copies the URL /callback?code=…&state=state-A, without opening it.

3. The attacker convinces the victim to open that URL.

4. The victim's browser requests /callback?code=…&state=state-A, but sends its own cookie (pre=V1) or none. It does not have the cookie pre=A1, which is in the attacker's browser.

5. The client compares state-A with the state associated with the victim's cookie. They do not match, and the client rejects the request.

6. The code is not exchanged and no session is created: the victim is not logged in to the attacker's account.

Conclusion

The attacker can put their state in the URL that the victim opens, but cannot put their cookie in the victim's browser. By requiring the state in the URL to match the browser's cookie, the client ensures that the login is completed by the same browser that started it. The same applies to the nonce: since it is stored in PENDING[pre], the expected nonce also becomes the one for this browser.

A.3 Lab 5 reflection

Question 1: In Lab 4 we came to trust an identity provider. In this lab, each host comes to trust the CA instead of each key. Both centralise authentication, and in both the new single point of failure is that centre, the IdP or the CA, because

compromising it gives access to everything that trusts it. The difference lies in timing: the IdP decides in real time and has to be available, whereas the CA signs in advance and the host decides on its own, without contacting it at login. That autonomy withstands the CA being unavailable, but makes revocation harder, since it is no longer central and comes to depend on a KRL distributed to each host.

Question 2: For an employee who leaves, the right answer is revocation: putting their active certificates in a KRL that the hosts load, and having the CA stop issuing new ones to them. The other three are not enough. Removing trust in the CA (authentication) would affect everyone, not only them. Removing their principal from the policy (authorisation) would require changing every host, and the certificate would remain valid wherever that was not done. Expiry would only lock them out when the certificate expired, and waiting on someone who has already left is not acceptable. Only revocation immediately ends a certificate that is still within its validity.

Question 3: The three attacks do not exploit bugs, but legitimate features left accessible by careless configuration. In the hijack, -A left on jump an agent socket accessible to root, which allowed signatures to be requested from the victim without ever having their key. In Lab 1, a world-writable cron file let a normal user run commands as root. The Docker socket exposed in a container gives full control of the host machine. In all of them, the pattern is the same: a useful feature exposed to whoever should not have it, a violation of least privilege, without any exploit.